

Delegates in C#



Delegates

Part-1

Mr. Bangar Raju

 **NARESH**
technologies

Understanding Delegates in C#: A Comprehensive Guide

By Albert Harmon

Published April 20, 2017

🕒 3 min read

Education



TLDR This blog post provides a detailed explanation of delegates in C#, including their definition, how to define and instantiate them, and how to call methods using delegates. It covers the similarities between delegates and method signatures, the importance of matching return types

and parameters, and the steps involved in using delegates effectively.

In the world of C#, delegates play a crucial role in enabling flexible method invocation. This blog post aims to demystify delegates, explaining what they are, how to define and instantiate them, and how to use them to call methods effectively.

What are Delegates?

A delegate in C# is defined as a type-safe function pointer. This means that a delegate holds a reference to a method and can invoke that method for execution. Essentially, it allows methods to be passed as parameters, enabling a more dynamic approach to method calls.

Calling Methods in C#

In C#, methods can be called in two primary ways:

1. **Using an instance of the class** for non-static methods.
2. **Using the class name** for static methods.

For example, to call a non-static method, you would first create an instance of the class and then call the method:

```
Program p = new Program();  
p.AddNums(100, 50);
```

For static methods, you can call them directly using the class name:

```
string result = Program.SayHello("John");
```

Calling Methods Using Delegates

Delegates provide an alternative way to call methods. To call a method using a delegate, you need to follow three steps:

Step 1: Define a Delegate

To define a delegate, you use the `delegate` keyword followed by the return type, delegate name, and parameter list. The syntax looks similar to a method signature:

```
public delegate void AddDelegate(int x, int y);
```

Here, the return type of the delegate must match the return type of the method it will reference, and the parameter types must also match.

Step 2: Instantiate the Delegate

Once the delegate is defined, the next step is to create an instance of the delegate. During instantiation, you pass the method name as a parameter to the delegate constructor:

```
AddDelegate addDelegate = new AddDelegate(p.AddNums);
```

If the method is static, you can reference it directly:

```
StringDelegate stringDelegate = new StringDelegate(Program.Say
```



Step 3: Call the Delegate

To call the method through the delegate, you simply invoke the delegate as if it were a method:

```
addDelegate(100, 50);  
string result = stringDelegate("John");
```

Alternatively, you can use the `Invoke` method provided by the delegate:

```
addDelegate.Invoke(100, 50);  
string result = stringDelegate.Invoke("John");
```

Important Points to Remember

- The return type of the delegate must match the return type of the method it references.
- The parameter types of the delegate must match the parameter types of the method.
- Delegates are reference types, similar to classes, and can be defined within a namespace or a class.
- Delegates can be instantiated to point to any method that matches their signature, allowing for flexible method invocation.

Conclusion

Delegates are a powerful feature in C# that enable type-safe method references and dynamic method invocation. By understanding how to define, instantiate, and call methods using delegates, developers can write more flexible and maintainable code. For further learning, consider exploring more advanced topics such as multicast delegates and events in C#.